Reviewer Pack — Cubical Euler Characteristic Bounds DNF-Min for Symmetric Fu…

Entry ed9920e4ad50 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-07 23:11:57 UTC

Cubical Euler Characteristic Bounds DNF-Min for Symmetric Functions

Entry ID: `ed9920e4ad50` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-07 23:11:57 UTC

1. Conjecture

Field A (mathematical branch): Cubical algebraic topology (Euler characteristic of subcomplexes of the n-cube) **Field B** (complexity object): DNF-min meta-complexity (gap-DNF-MCSP) for symmetric Boolean functions

Statement:

> For every symmetric Boolean function f:{0,1}^n→{0,1} (n≥2), determined by its on-weight set R_f:={k∈{0,…,n}: f(any weight-k input)=1}, define the cubical complex K_f as the union of all faces C of the n-cube on which f≡1, and let χ(K_f) be its Euler characteristic, computed by the closed form χ(K_f)=Σ_{d=0}^{n}(-1)^d·C(n,d)·Σ_{w_0:[w_0,w_0+d]⊆R_f} C(n-d,w_0). Then DNF-min(f) ≥ |χ(K_f)|, with equality whenever R_f consists only of isolated weights (so the bound is saturated by parity, where both equal 2^{n-1}). One symmetric f with DNF-min(f) < |χ(K_f)| falsifies the conjecture.

Rationale (proposer's reasoning):

> Each minterm of a DNF for f is a face of K_f, so χ(K_f) measures the inclusion–exclusion residue across all such faces; a small DNF has few subcubes available to cancel into χ, forcing |χ| to be small. Restricting to symmetric f (a 2^{n+1}-size family) sidesteps the natural-proofs barrier — the discriminating property is not 'large' on the full 2^{2^n} function space, evading Razborov–Rudich pseudo-random-function obstructions while still using a constructive, poly(2^n)-computable invariant. The bridge between cubical Euler characteristic and exact DNF-minimization for the symmetric class appears nowhere in the standard literature on MCSP, set-cover, or AC^0 lower bounds.

Taxonomy category: `META_COMPLEXITY` (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `f6a48790b8f86d8f`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across n∈{4,5,6,7,8,10} with 30 seeds × 30 random R each (and exhaustive enumeration of all 2^{n+1} subsets for n≤7), every sampled symmetric f must satisfy DNF-min(f) ≥ |χ(K_f)|; additionally, equality must hold for every R consisting solely of isolated weights (no two consecutive).

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.90	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - `symmetric Boolean functions DNF minimization Euler characteristic` - `cubical complex subcomplex hypercube Boolean function complexity` - `minimum DNF symmetric functions lower bound topological`

Top relevant hits considered: - [http://arxiv.org/abs/2306.14401v1] On the distribution of sensitivities of symmetric Boolean functions - [http://arxiv.org/abs/1805.08177v4] A Polynomial Time Delta-Decomposition Algorithm for Positive DNFs - [http://arxiv.org/abs/0808.0684v1] 9-variable Boolean Functions with Nonlinearity 242 in the Generalized Rotation Class - [http://arxiv.org/abs/2605.00705v1] Cohomological properties of the Vietoris–Rips Complex of a Hypercube Graph - [http://arxiv.org/abs/1407.6169v3] Multiplicative Complexity of Vector Valued Boolean Functions - [http://arxiv.org/abs/2505.23318v2] Minimal displacement set for CAT(0) cubical complexes - [http://arxiv.org/abs/2509.02841v2] Toward Lower Bounds for Chromatic Symmetric Functions in the Elementary Basis - [http://arxiv.org/abs/1110.6876v4] Lower bounds for topological complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=0.1s

5.1 Generated Python source

```python
import random
from itertools import combinations, chain

def binomial(n, k):
    if k > n:
        return 0
    result = 1
    for i in range(k):
        result *= (n - i)
        result //= (i + 1)
    return result

def gaussian_elimination(A):
    m, n = len(A), len(A[0])
```

```
    rank = 0
    for j in range(n):
        pivot_row = None
        for i in range(rank, m):
            if A[i][j] != 0:
                pivot_row = i
                break
        if pivot_row is not None:
            A[pivot_row], A[rank] = A[rank], A[pivot_row]
            for i in range(m):
                if i != rank and A[i][j] != 0:
                    factor = A[i][j] / A[rank][j]
                    for k in range(n):
                        A[i][k] -= factor * A[rank][k]
            rank += 1
    return rank

def matrix_multiplication(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C

def determinant(A):
    m, n = len(A), len(A[0])
    if m != n:
        raise ValueError("Matrix must be square")
    if n == 1:
        return A[0][0]
    det = 0
    for j in range(n):
        submatrix = [row[:j] + row[j+1:] for row in A[1:]]
        sign = (-1) ** j
        det += sign * A[0][j] * determinant(submatrix)
    return det

def powerset(s):
    result = []
    for r in range(len(s) + 1):
        result.extend(combinations(s, r))
    return result

def symmetric_function(n, R):
    def f(x):
        weight = sum(1 for bit in x if bit == '1')
        return weight in R
    return f

def dnf_min(f):
    n = len(next(iter(f.keys())))
    truth_table = ['0' * n + '1'] + [''.join(p) + '1' for p in powerset(range(n))]
    prime_implicants = []
    for minterm in truth_table:
```

```
        if f[minterm]:
            covered = False
            for pi in prime_implicants:
                if all(minterm[i] == '0' or minterm[i] == pi[i] for i in range(n)):
                    covered = True
                    break
            if not covered:
                prime_implicants.append(minterm)
    return len(prime_implicants)

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n_values = [4, 5, 6, 7, 8, 10]
    metric_name = "DNF-min vs |χ(K_f)|"
    instances_tested = 0
    conjecture_holds = True
    counterexample = ""

    for n in n_values:
        R_samples = set()
        if n <= 7:
            R_samples.update(powerset(range(n + 1)))
        else:
            for _ in range(30):
                R = random.sample(range(n + 1), random.randint(1, n))
                R_samples.add(tuple(sorted(R)))

        for R in R_samples:
            f = symmetric_function(n, set(R))
            K_f = []
            for i in range(2**n):
                binary_rep = format(i, '0{}b'.format(n))
                if all(f[binary_rep[:j] + '1'] for j in range(len(binary_rep))):
                    K_f.append(binary_rep)

            chi_K_f = 0
            for d in range(n + 1):
                C_n_d = binomial(n, d)
                w_0s = [w for w in R if w >= d]
                chi_K_f += (-1)**d * C_n_d * sum(binomial(n - d, w_0) for w_0 in w_0s)

            dnf_min_f = dnf_min(f)
            instances_tested += 1

            if dnf_min_f < abs(chi_K_f):
                conjecture_holds = False
                counterexample = f"n={n}, R={R}"
                break

    return {
        "metric_name": metric_name,
        "metric_value": chi_K_f,
        "instances_tested": instances_tested,
        "conjecture_holds": conjecture_holds,
        "counterexample": counterexample
    }
```

```python
if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 30)) + list(range(53
    results = []

    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")

    mean_metric_value = sum(result["metric_value"] for result in results) / len(results)
    std_metric_value = (sum((result["metric_value"] - mean_metric_value)**2 for result in results)
    support_fraction = sum(1 for result in results if result["conjecture_holds"]) / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std={std_metric_value} support_fractior
    elif any(not result["conjecture_holds"] for result in results):
        first_failing_seed = next(seed for seed, result in enumerate(results) if not result["conje
        print(f"RESULT: FALSIFIED counterexample=\"{result['counterexample']}\" first_failing_seed
    else:
        print("RESULT: INCONCLUSIVE mapping_undefined")
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e3099eab.py", line 147, in <module>
    result = run_trial(seed)
             ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e3099eab.py", line 116, in run_trial
    if all(f[binary_rep[:j] + '1'] for j in range(len(binary_rep))):
       ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_e3099eab.py", line 116, in <genexpr>
    if all(f[binary_rep[:j] + '1'] for j in range(len(binary_rep))):
           ~^^^^^^^^^^^^^^^^^^^^^^
TypeError: 'function' object is not subscriptable
```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

> The test crashed with a TypeError ('function' object is not subscriptable) before producing any data, so neither support nor falsification conditions were evaluated. | next: Fix

the test harness bug (treat f as a callable, not a dict — replace f[binary_rep[:j]+'1'] with f(binary_rep[:j]+'1') or precompute f as a dict) and rerun the pre-registered protocol.

11. Audit log (LLM calls)

Total LLM calls: 7

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	380461
2	preregistration	claude_max	opus	0	0	7250
3	novelty	claude_max	opus	0	0	4486
4	novelty	claude_max	opus	0	0	8832
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	21098
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	35826
7	judge	claude_max	opus	0	0	5139

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 463092 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 2}

(full prompt+response transcripts available in `research/audit/ed9920e4ad50.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/ed9920e4ad50.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/ed9920e4ad50.tar.gz` (if generated)